

ui.button_group 全面详解（基于 NiceGUI 文档）

ui.button_group 是 NiceGUI 中用于对多个按钮（或按钮类组件）进行分组管理的容器组件，基于 Quasar 的 QBtnGroup 组件实现。其核心价值在于统一控制组内组件的样式、布局和交互逻辑，让相关操作按钮形成视觉与功能的聚合，提升界面的规整性和用户体验。以下从核心特性、基础用法、样式定制、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心规则

- 底层依赖：基于 Quasar 的 QBtnGroup 组件，继承其分组布局和样式统一能力。
- 关键约束：**父组件 (button_group) 与子组件 (按钮、下拉按钮等) 必须使用相同的设计属性 (props)**，否则会导致样式错乱（如分组边框断裂、间距异常）。
- 支持子组件类型：不仅支持 ui.button，还可嵌套 ui.dropdown_button 等按钮类组件，实现复杂分组交互。

2. 核心属性

属性名	类型	说明
classes	Classes[Self]	组件的 CSS 类（支持 Tailwind/Quasar 样式类，用于统一调整分组布局）
client	Client	组件所属的客户端实例
html_id	str	HTML 元素的唯一 ID（2.16.0+ 版本新增，用于 DOM 操作定位）
is_deleted	bool	组件是否已被删除（只读状态）
is_ignoring_events	bool	组件是否正在忽略事件（只读状态）
parent_slot	Slot None	组件的父插槽（可设置，用于调整组件在父容器中的插入位置）
props	Props[Self]	组件的 Quasar 特性属性（核心样式控制入口，需与子组件保持一致）
style	Style[Self]	组件的内联 CSS 样式（用于精细化调整分组外观）
visible	BindableProperty	组件的可见性（可绑定数据动态控制，支持双向绑定）

二、基础使用示例

1. 最简按钮组（基础交互）

将多个独立按钮封装在 ui.button_group() 上下文管理器中，自动实现横向紧凑排列和视觉分组：

```

from nicegui import ui

with ui.button_group():
    # 组内按钮各自绑定点击事件
    ui.button('One', on_click=lambda: ui.notify('You clicked Button 1!'))
    ui.button('Two', on_click=lambda: ui.notify('You clicked Button 2!'))
    ui.button('Three', on_click=lambda: ui.notify('You clicked Button 3!'))

ui.run()

```

效果：三个按钮横向紧密排列，形成统一视觉组，点击各自触发对应通知。

2. 包含下拉按钮的混合分组

支持嵌套 `ui.dropdown_button`，实现“普通按钮 + 下拉菜单”的复合分组，适用于操作选项较多的场景：

```

from nicegui import ui

with ui.button_group():
    ui.button('One', on_click=lambda: ui.notify('Button One'))
    ui.button('Two', on_click=lambda: ui.notify('Button Two'))
    # 下拉按钮作为组内最后一个组件
    with ui.dropdown_button('Dropdown'):
        ui.item('Item 1', on_click=lambda: ui.notify('Dropdown Item 1'))
        ui.item('Item 2', on_click=lambda: ui.notify('Dropdown Item 2'))
        ui.item('Item 3', on_click=lambda: ui.notify('Dropdown Item 3'))

ui.run()

```

效果：前两个为普通按钮，第三个为下拉按钮，整体保持分组样式一致性，下拉菜单展开后不破坏分组布局。

三、样式定制（核心特性）

`ui.button_group` 的样式定制核心是「父组件 props 与子组件 props 统一」，支持 Quasar 按钮的所有设计属性（如 `rounded`、`outline`、`push` 等），以下是常见样式示例：

1. 圆角按钮组

通过 `props('rounded')` 为分组和子按钮设置圆角样式：

```

from nicegui import ui

# 父组件设置 rounded 属性
with ui.button_group().props('rounded'):
    # 子按钮必须同步设置 rounded，保持样式一致
    ui.button('One').props('rounded')
    ui.button('Two').props('rounded')
    ui.button('Three').props('rounded')

ui.run()

```

2. 按压式 (Push) 按钮组

`push` 属性让按钮点击时呈现“按压凹陷”效果，配合 `glossy` 可增加光泽感，子按钮支持自定义颜色：

```
from nicegui import ui

# 父组件设置 push + glossy 特性
with ui.button_group().props('push glossy'):
    # 子按钮同步 push 属性，各自设置颜色（部分按钮需调整文本颜色以适配背景）
    ui.button('One', color='red').props('push')
    ui.button('Two', color='orange').props('push text-color=black')
    ui.button('Three', color='yellow').props('push text-color=black')

ui.run()
```

3. 轮廓式 (Outline) 按钮组

`outline` 属性让按钮仅显示边框，无填充背景，适合轻量化界面：

```
from nicegui import ui

# 父组件设置 outline 特性
with ui.button_group().props('outline'):
    # 子按钮必须同步 outline 属性
    ui.button('One').props('outline')
    ui.button('Two').props('outline')
    ui.button('Three').props('outline')

ui.run()
```

4. 全局默认样式修改

通过 `default_classes`、`default_props`、`default_style` 方法，为所有 `ui.button_group` 实例设置全局默认样式（需在组件实例化前调用）：

```
from nicegui import ui

# 全局默认：所有按钮组添加圆角和内边距
ui.button_group.default_props(add='rounded')
ui.button_group.default_classes(add='p-2')

# 后续创建的按钮组自动继承默认样式
with ui.button_group():
    ui.button('One').props('rounded') # 仍需同步子组件 props
    ui.button('Two').props('rounded')
    ui.button('Three').props('rounded')

with ui.button_group():
    ui.button('A').props('rounded')
    ui.button('B').props('rounded')
```

```
ui.run()
```

四、核心方法与功能扩展

1. 可见性控制与绑定

- 直接控制：通过 `set_visibility(visible: bool)` 手动设置按钮组显示 / 隐藏：

```
from nicegui import ui

group = ui.button_group()
with group:
    ui.button('One')
    ui.button('Two')

# 隐藏按钮组
group.set_visibility(False)

# 1 秒后显示
ui.timer(1, lambda: group.set_visibility(True))

ui.run()
```

- 数据绑定：通过 `bind_visibility` 系列方法，将可见性与目标对象的属性绑定（支持单向 / 双向绑定）：

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.show_group = True

state = AppState()

# 双向绑定：按钮组可见性与 state.show_group 同步
with ui.button_group().bind_visibility(state, 'show_group'):
    ui.button('One')
    ui.button('Two')

# 开关控制绑定属性，间接控制按钮组显示
ui.switch('Show Button Group', value=state.show_group).bind_value(state, 'show_group')

ui.run()
```

2. 组件操作方法

方法名	说明	参数示例
<code>clear()</code>	移除组内所有子组件	<code>group.clear()</code>
<code>delete()</code>	删除按钮组及所有子组件	<code>group.delete()</code>
<code>remove(element)</code>	移除指定子组件（支持元素实例或 ID）	<code>group.remove(button_one)</code>

方法名	说明	参数示例
<code>move(target_container)</code>	移动按钮组到其他容器	<code>group.move(ui.card())</code>
<code>tooltip(text)</code>	为整个按钮组添加悬停提示	<code>group.tooltip('操作组')</code>
<code>update()</code>	触发客户端界面更新（修改属性后需调用）	<code>group.update()</code>

3. 事件订阅

通过 `on(type: str, handler)` 订阅按钮组的 DOM 事件（如 `click`、`mousedown` 等），需注意事件会冒泡，若子组件绑定了相同事件，会优先触发子组件事件：

```
from nicegui import ui

with ui.button_group() as group:
    group.on('click', lambda: ui.notify('Button Group Clicked!'))
    ui.button('One') # 未绑定自身点击事件，点击时触发组的 click 事件
    ui.button('Two', on_click=lambda: ui.notify('Button Two Clicked!')) # 优先触发自身事件

ui.run()
```

五、使用场景与注意事项

1. 适用场景

- 功能关联操作：如“新建 / 编辑 / 删除”、“上一步 / 下一步 / 完成”等流程化操作。
- 选项切换：如数据排序方式（升序 / 降序）、视图切换（列表 / 网格）等互斥或关联选项。
- 复合操作入口：结合下拉按钮，收纳次要操作，保留核心按钮在组内。

2. 关键注意事项

- 样式一致性：**父组件与子组件必须使用相同的设计 props**（如 `rounded`、`outline`），否则会出现边框错位、间距异常等问题。
- 子组件类型限制：仅支持按钮类组件（`ui.button`、`ui.dropdown_button`），嵌套非按钮组件（如 `ui.label`、`ui.image`）会破坏分组样式。
- 事件冒泡：按钮组的事件会被子组件的同名事件覆盖，若需同时触发，需在子组件事件中手动调用组的事件处理逻辑。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_visibility` 的 `strict` 参数在 3.0.0+ 版本支持，使用时需注意版本匹配。

六、进阶示例：动态增减组内按钮

结合 `clear()`、`remove()` 方法和按钮事件，实现动态添加 / 删除组内按钮的功能：

```
from nicegui import ui

with ui.column():
```

```

# 创建按钮组并赋值给变量，用于后续操作
button_group = ui.button_group()
with button_group:
    btn1 = ui.button('One', on_click=lambda: ui.notify('One'))
    btn2 = ui.button('Two', on_click=lambda: ui.notify('Two'))

# 动态添加按钮
def add_button():
    new_btn = ui.button(f'New {len(button_group.children) + 1}',
                        on_click=lambda: ui.notify(f'New Button Clicked'))
    button_group.add(new_btn)
    # 同步父组件 props（假设父组件为 rounded 样式）
    new_btn.props('rounded')
    button_group.update()

# 动态移除最后一个按钮
def remove_last_button():
    if len(button_group.children) > 0:
        button_group.remove(button_group.children[-1])
        button_group.update()

# 控制按钮
ui.button('Add Button', on_click=add_button)
ui.button('Remove Last', on_click=remove_last_button)

ui.run()

```

效果：点击“Add Button”可添加新按钮（自动同步分组样式），点击“Remove Last”可删除最后一个按钮，按钮组实时更新布局。